

Chapter 14

Tool Calling in LangChain

How an LLM connects to a tool, decides when to use it, and how the programmer safely executes it — culminating in a real-time currency conversion application.

WHAT THIS CHAPTER COVERS

- ▶ Tool Binding with `.bind_tools()`
- ▶ How an LLM decides to call a tool
- ▶ Anatomy of a structured `tool_call`
- ▶ Tool Execution and `ToolMessage`
- ▶ Handling dependent tool arguments
- ▶ `InjectedToolArg` annotation
- ▶ Building a real-time currency converter
- ▶ Manual orchestration vs. AI Agents

Tool Calling in LangChain

RECAP — CHAPTER 13 (TOOLS IN LANGCHAIN)

The previous chapter introduced *Tools*: special Python functions wrapped with the `@tool` decorator that give an LLM the ability to act, not just talk. A tool carries a name, a description, and an input schema so the LLM knows what it does and how to call it. What that chapter deliberately left open was **how** an LLM and a tool actually get connected, and **how** the LLM decides when to trigger one. That missing piece — Tool Binding and Tool Calling — is exactly what this chapter covers.

1. Tool Binding

Definition

Tool Binding is the step where one or more tools are registered with a Large Language Model so that the LLM becomes aware of: (1) which tools are available to it, (2) what each tool does, and (3) what input format each tool expects.

Why It Exists

An LLM only knows about a tool if it has explicitly been told about it. Without binding, the model has no way of knowing a `multiply` function or a currency-conversion function even exists, no matter how well the function itself is written.

How It Works

Every model object created through LangChain exposes a `.bind_tools()` method. Passing a list of tool objects into this method returns a new, augmented LLM object that carries tool-awareness. The original LLM object is left untouched; the result of binding must be stored in a new variable.

1. Define a tool using the `@tool` decorator (name, docstring/description, typed parameters).
2. Create a base chat model (e.g. `ChatGroq`, `ChatOpenAI`).
3. Call `llm.bind_tools([tool_1, tool_2, ...])` and store the result, conventionally as `llm_with_tools`.

```

from langchain_huggingface import ChatHuggingFace, HuggingFaceEndpoint
from langchain_core.tools import tool

# 1. Tool creation
@tool
def multiply(a: int, b: int) -> int:
    """Given two numbers a and b this tool returns their product"""
    return a * b

# 2. Tool binding — llm_with_tools now knows this tool exists
llm = ChatHuggingFace(llm=HuggingFaceEndpoint(repo_id="Qwen/Qwen2.5-7B-Instruct"))
llm_with_tools = llm.bind_tools([multiply])

```

The last line is the entire mechanics of tool binding — a single method call that returns an LLM object with tool awareness baked in.

COMMON MISTAKE

Calling `llm.bind_tools([...])` without assigning the result to a new variable and then continuing to invoke the original `llm`. The original object is unmodified — only `llm_with_tools` (or whatever the result is stored as) actually has tool access.

IMPORTANT

Not every LLM supports tool binding. Only models that expose function-calling / tool-calling capability at the API level can be bound to tools — this is a model-provider feature, not something LangChain can add to a model that lacks it.

Quick Revision — Tool Binding

- Tool Binding registers a tool with an LLM so the model knows the tool's existence, purpose, and input schema.
- Achieved with the `.bind_tools([tool_1, tool_2, ...])` method available on chat model objects.
- The result must be stored in a new variable — binding does not mutate the original LLM object.
- Multiple tools can be bound at once by listing them inside `bind_tools()`.
- Only LLMs whose provider API supports function/tool calling can be bound to tools.

2. Tool Calling

Definition

Tool Calling is the process where, during a conversation, an LLM decides that it needs to use a specific tool to answer the current query, and generates a structured output containing the tool's name and the arguments to call it with.

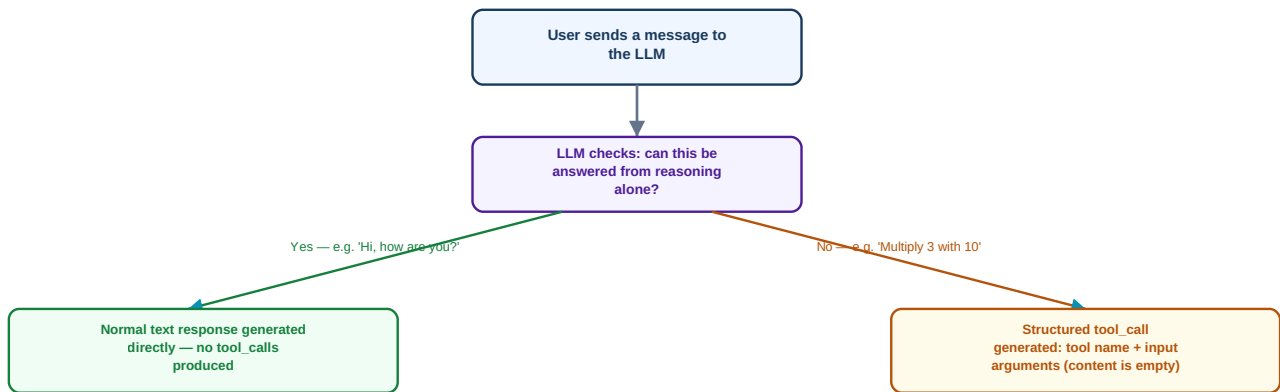
Why It Exists

An LLM bound to tools does not blindly call a tool on every message. It must first reason about whether the current query even requires external action. Tool calling is the decision-and-signalling mechanism that separates "just answer in text" from "I need help from a tool."

How It Works

When a tool-bound LLM is invoked with a message:

1. If the query can be answered from the model's own reasoning/knowledge (e.g. "Hi, how are you?"), the LLM responds with normal text in `AIMessage.content` and produces no tool call.
2. If the query matches something a bound tool can help with (e.g. "Can you multiply 3 with 10?"), the LLM leaves `content` empty and instead populates an `AIMessage.tool_calls` list with one or more structured tool call objects.



The LLM only ever suggests the tool and arguments — it never executes the tool itself.

Figure 14.1 — How a tool-bound LLM decides whether to respond with text or a tool call.

ANATOMY OF A TOOL_CALL

Each entry inside `AIMessage.tool_calls` is a dictionary with four keys:

Key	Meaning
name	The name of the tool the LLM believes is relevant to the query.
args	A dictionary of the input arguments the LLM has inferred from the query, matching the tool's input schema.
id	A unique identifier for this particular tool call.
type	Always the literal string "tool_call".

```
from langchain_core.messages import HumanMessage

messages = [HumanMessage("Can you multiply 3 with 10")]
ai_message = llm_with_tools.invoke(messages)

# content is empty; the actual instruction lives in tool_calls
print(ai_message.tool_calls)
# [{'name': 'multiply', 'args': {'a': 3, 'b': 10}, 'id': '...', 'type': 'tool_call'}]
```

IMPORTANT

The LLM does **not** actually run the tool during tool calling. It only *suggests* the tool and the arguments to call it with. The real execution is always handled separately, by LangChain and the programmer.

INTERVIEW TIP

Be precise when asked to explain tool calling: the correct answer is that the LLM produces a structured suggestion (tool name + arguments), never that it "runs" or "calls" the tool directly. This distinction is a very common interview trap.

Quick Revision — Tool Calling

- Tool calling is the LLM deciding, per query, whether a bound tool is needed.
- If no tool is needed, the LLM answers normally through `AIMessage.content`.
- If a tool is needed, `content` is empty and `AIMessage.tool_calls` is populated instead.
- Each tool call has a `name`, `args`, a unique `id`, and `type: 'tool_call'`.
- The LLM never executes the tool itself — it only produces a recommendation.

3. Tool Execution

Definition

Tool Execution is the step where the actual Python function behind a tool is run, using the exact input arguments that the LLM suggested during tool calling.

Why It Exists

Since the LLM only recommends a tool call and never executes it, someone has to actually run the underlying function and feed the result back into the conversation. That responsibility belongs to the programmer (or, later, to an autonomous agent).

How It Works

A tool call dictionary can be passed directly into a tool's `.invoke()` method. Passing the arguments alone (`tool.invoke(tool_call["args"])`) returns just the raw return value. Passing the **entire** tool call dictionary (`tool.invoke(tool_call)`) returns a properly wrapped `ToolMessage` instead — the format required to send the result back to the LLM.

```
from langchain_core.messages import HumanMessage

query = HumanMessage("Can you multiply 2 with 4")
messages = [query]

# 3. Tool calling
call = llm_with_tools.invoke(messages)
messages.append(call) # store the AIMessage in the running history

# 4. Tool execution — pass the full tool_call to get a ToolMessage back
tool_result = multiply.invoke(call.tool_calls[0])
messages.append(tool_result) # ToolMessage is added to history

# Send the full conversation (Human + AI + Tool) back to the LLM
final_result = llm_with_tools.invoke(messages)
print(final_result.content) # "The product of 2 and 4 is 8."
```

WHY A TOOLMESSAGE MATTERS

A `ToolMessage` is a fourth message type (alongside `HumanMessage`, `AIMessage`, and `SystemMessage`) produced specifically when a tool is executed using its associated tool call. Feeding this `ToolMessage` back into the running `messages` list gives the LLM full context of what happened, so its final natural-language reply is grounded in the real tool output rather than a guess.

BEST PRACTICE

Always maintain a running `messages` list — Human, then AI, then Tool — and re-invoke the LLM with the complete history at the end. Sending only the original hard-coded query for the final call (instead of the full `messages` list) silently drops the tool result and produces a stale or generic answer.

🔄 Quick Revision — Tool Execution

- Tool execution means actually running the tool's Python function with the LLM-suggested arguments.
- `tool.invoke(tool_call["args"])` returns the raw value only.
- `tool.invoke(tool_call)` (the full dictionary) returns a properly formatted `ToolMessage`.
- `ToolMessage` is a distinct message type alongside `HumanMessage`, `AIMessage`, and `SystemMessage`.
- The full message history (Human, AI, Tool) must be sent back to the LLM to get a grounded final answer.

4. Handling Dependent Tool Arguments — InjectedToolArg

Problem It Solves

When a single user query requires **two chained tool calls** where the second tool needs a value produced by the first (e.g. get a conversion rate, then use that rate to convert an amount), the LLM tends to generate *both* tool calls in one pass — before the first tool has actually run. Since the second tool call is generated too early, the LLM has no real value for the dependent argument and falls back on its own stale parametric knowledge, silently producing an incorrect result.

COMMON MISTAKE

Assuming that because the LLM produced two tool calls together, they will automatically be executed in the correct dependent order with real intermediate values. In practice the LLM may guess a plausible-looking but outdated value (e.g. an old USD-to-INR rate from its training data) for an argument it was never actually given.

How It Works

LangChain solves this with `InjectedToolArg`: an annotation that marks a specific tool parameter as one the LLM should never try to fill in itself. The developer is responsible for injecting the real value manually, after the dependency tool has actually executed.

```
from langchain_core.tools import InjectedToolArg
from typing import Annotated

@tool
def convert(base_currency_amount: int,
            conversion_factor: Annotated[float, InjectedToolArg]) -> float:
    """ Multiplies the amount by the conversion factor to get the converted value. """
    return base_currency_amount * conversion_factor
```

With this annotation in place, the LLM will only populate `base_currency_amount` in its generated tool call for `convert` — `conversion_factor` is deliberately left out. The programmer is then free to insert the correct, freshly-fetched value into `tool_call["args"]` before invoking the tool.



Figure 14.2 — Why a dependent argument must be marked `InjectedToolArg` rather than left for the LLM to guess.

IMPORTANT

`InjectedToolArg` does not fetch or compute the value for you — it only tells the LLM "do not try to fill this in." The actual assignment (`tool_call["args"]["conversion_factor"] = conversion_factor`) is still explicit code the developer writes.

Quick Revision — InjectedToolArg

- Chained tool calls generated together can cause the LLM to guess a dependent argument's value.
- `InjectedToolArg` (with `typing.Annotated`) marks a parameter the LLM must never try to fill in.
- The developer manually injects the real value into `tool_call['args']` after the dependency tool runs.
- This prevents the LLM from substituting stale parametric-knowledge values for real-time data.

5. Building a Real Application — Real-Time Currency Converter

Why It Exists

An LLM's knowledge of exchange rates is frozen at its training cut-off, so asking it directly for a currency conversion returns a stale, historical figure rather than the live rate. This application solves that by giving the LLM access to a real-time exchange rate API through two chained tools.

How It Works

The application uses two tools working together:

- **get_conversion_factor** — hits a live exchange-rate API with a base and target currency and returns the current conversion rate.
- **convert** — multiplies a given base-currency amount by the conversion factor (marked as an `InjectedToolArg`, since it depends on the first tool's output).

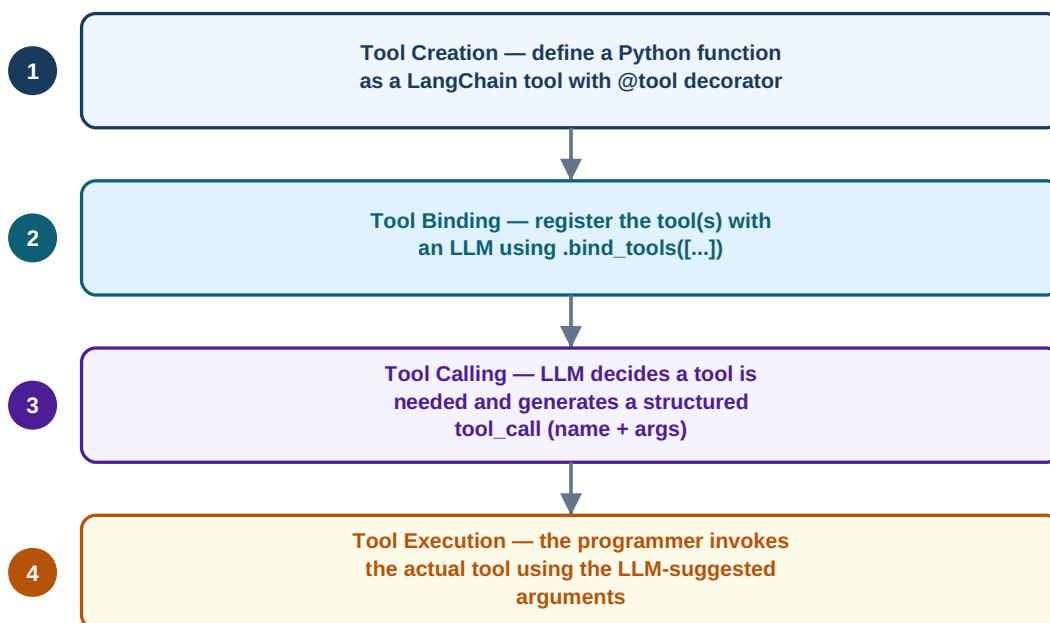


Figure 14.3 — The complete four-stage flow this application walks through, end to end.


```

from langchain_core.tools import tool
from langchain_core.messages import HumanMessage
from langchain_core.tools import InjectedToolArg
from typing import Annotated
from langchain_groq import ChatGroq
import requests
import json
from dotenv import load_dotenv

load_dotenv()

@tool
def get_conversion_factor(base_currency: str, target_currency: str) -> float:
    """ Fetches the conversion factor between a base currency and a target currency. """
    # API key is loaded from the environment – never hard-code real keys in source
    url = f"https://v6.exchangerate-api.com/v6/<YOUR_API_KEY>/pair/{base_currency}/{target_currency}"
    response = requests.get(url)
    return response.json()["conversion_rate"]

@tool
def convert(base_currency_amount: int,
            conversion_factor: Annotated[float, InjectedToolArg]) -> float:
    """ Uses the conversion factor to convert the base amount into the target currency. """
    return base_currency_amount * conversion_factor

# Tool Binding
llm = ChatGroq(model="llama-3.3-70b-versatile")
llm_with_tools = llm.bind_tools([get_conversion_factor, convert])

```

BEST PRACTICE

Never commit a real API key inside source code. The reference implementation loads secrets through `load_dotenv()` and an environment variable; the placeholder `<YOUR_API_KEY>` shown above should be replaced with a value read from your own `.env` file, never hard-coded inline.

REAL WORLD INSIGHT

The exchange-rate API used here returns the live conversion rate along with a last-updated timestamp. Rates for major currency pairs typically refresh on an hourly basis, which is precisely why a static, training-time answer from the LLM alone would be unreliable for this use case.

DRIVING THE CONVERSATION AND EXECUTING BOTH TOOLS

```
messages = [HumanMessage(
    "What is the conversion factor between USD and INR, and based on that, "
    "can you convert 10 USD to INR"
)]

# Tool calling – LLM proposes two tool calls
ai_message = llm_with_tools.invoke(messages)
messages.append(ai_message)

for tool_call in ai_message.tool_calls:

    # Execute the 1st tool and obtain the real conversion factor
    if tool_call['name'] == "get_conversion_factor":
        tool_message1 = get_conversion_factor.invoke(tool_call)
        conversion_factor = json.loads(tool_message1.content)["conversion_rate"]
        messages.append(tool_message1)

    # Inject the fetched conversion_factor before executing the 2nd tool
    if tool_call["name"] == "convert":
        tool_call["args"]["conversion_factor"] = conversion_factor
        tool_message2 = convert.invoke(tool_call)
        messages.append(tool_message2)

# Final call – LLM now has Human + AI + both Tool messages as context
print(llm_with_tools.invoke(messages).content)
```

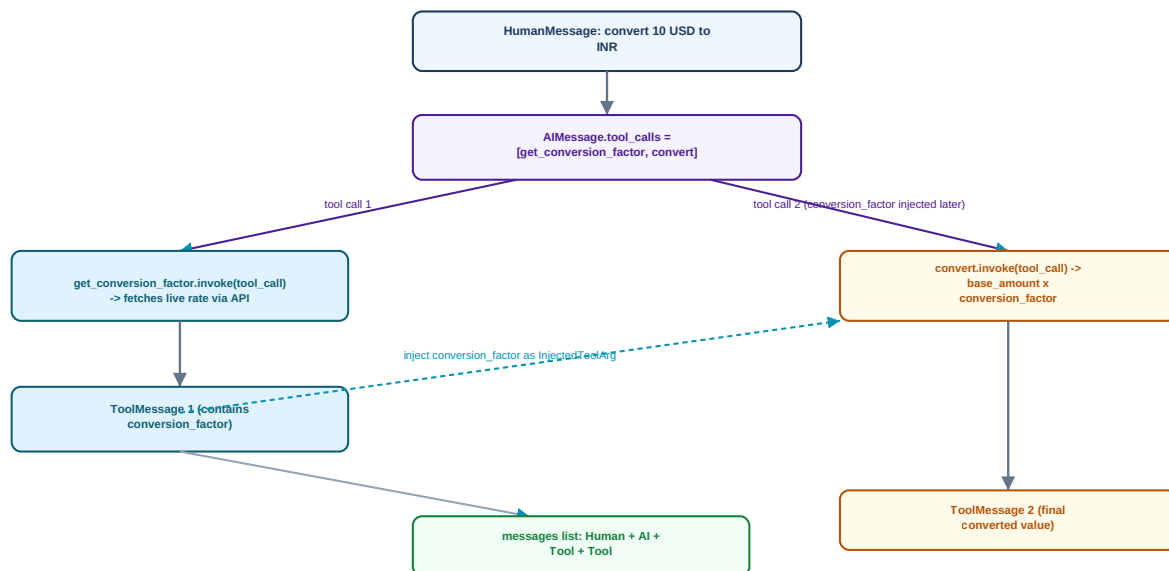


Figure 14.4 — Sequential execution of two chained tools, with the conversion factor injected between them.

COMMON MISTAKE

Asking the LLM two related sub-questions in a single message (rate lookup + conversion) causes it to generate both tool calls simultaneously, before either has run. The second tool call's dependent argument must therefore always be filled manually — never trust the LLM's own guess for it.

🔄 Quick Revision — Currency Conversion Application

- The app combines two tools: `get_conversion_factor` (live API lookup) and `convert` (multiplication).
- `conversion_factor` is an `InjectedToolArg` because `convert` depends on `get_conversion_factor`'s result.
- A single compound query can generate multiple `tool_calls` in one `AIMessage` at once.
- The programmer loops over `tool_calls`, executes each tool, and injects dependent values manually.
- The full conversation history is replayed to the LLM at the end to produce a grounded final answer.

6. Is This an AI Agent?

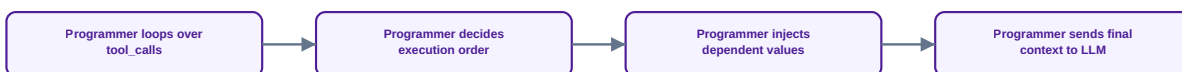
Definition

Despite using tools, tool binding, and tool calling, the currency-conversion application built in this chapter is **not** an AI agent. It is a manually orchestrated tool-using LLM application.

Why It Matters

The defining characteristic of an AI agent is *autonomy*: the ability to break a problem into steps and work through them sequentially, deciding at each step what to do next, without a human writing the step-by-step control flow. In this chapter's application, the programmer — not the model — decided which tool call to execute first, extracted the conversion factor, injected it into the second call, and finally resent the full context to the LLM.

MANUAL ORCHESTRATION (what this chapter built)



AUTONOMOUS AI AGENT (next chapter)



Key difference: *autonomy*. An agent runs this whole loop itself, without a human writing the glue code.

Figure 14.5 — Manual orchestration (this chapter) versus what an autonomous agent would do instead.

Aspect	Manual Orchestration (this chapter)	Autonomous AI Agent
Decides tool order	Programmer, via explicit code	Agent decides dynamically at runtime
Injects dependent values	Programmer writes the injection code	Agent chains results between its own tool calls
Control flow	Fixed loop written in advance	Emerges from the agent's own reasoning loop
Human involvement mid-task	High — every step is hand-coded	Low — agent runs the loop itself

REAL WORLD INSIGHT

Tools, tool binding, and tool calling are the essential building blocks agents are built from — but stacking them together manually with hard-coded control flow is not yet agentic behaviour. The next step in the roadmap is replacing this hand-written loop with an autonomous agent that performs the same reasoning-and-tool-chaining process on its own.

🔄 Quick Revision — Agent vs. Manual Orchestration

- Using tools, binding, and tool calling does not automatically make an application an AI agent.
- Autonomy — deciding steps and order without hand-written control flow — is the defining trait of an agent.
- This chapter's app used programmer-written loops and manual value injection, not autonomous decision-making.
- An agent would break the same problem into steps and chain tool results together on its own.
- This distinction sets up the transition to building true AI agents in the next chapter.

K. Key Takeaways

- **Tool Binding** registers one or more tools with an LLM via `.bind_tools([...])`, giving the model awareness of each tool's name, purpose, and input schema.
- **Tool Calling** is the LLM's decision, per query, to request a tool by producing a structured `tool_calls` entry instead of plain text — it never executes the tool itself.
- **Tool Execution** is the programmer (or agent) actually invoking the tool's underlying function with the LLM-suggested arguments, producing a `ToolMessage`.
- **ToolMessage** is a fourth core message type, alongside Human, AI, and System messages, and must be appended to the running conversation for the LLM to produce a grounded final answer.
- **InjectedToolArg** marks a dependent tool parameter that the LLM must not try to fill in itself, letting the developer inject real intermediate values manually.
- **The currency conversion application** demonstrated chaining two tools — a live rate lookup and a multiplication step — using `InjectedToolArg` to pass real data between them.
- **Manual orchestration is not the same as an AI agent** — true agents are distinguished by autonomy in deciding steps and tool order, which is the subject of the next chapter.

R. Revision Sheet

Tool Binding

Registering tools with an LLM via `.bind_tools([...])` so the model knows what tools exist, what they do, and their input format.

Tool Calling

The LLM's structured suggestion of a tool name and arguments (via `AIMessage.tool_calls`) — it never runs the tool itself.

Tool Execution

Actually invoking the tool with the LLM's suggested arguments, typically via `tool.invoke(tool_call)`, producing a `ToolMessage`.

ToolMessage

A message type carrying a tool's execution result back into the conversation so the LLM can ground its final reply in real data.

InjectedToolArg

An `Annotated` marker telling the LLM to skip filling a specific argument, which the developer injects manually instead.

Manual Orchestration vs. Agent

Hand-written loops that call tools in a fixed order are not autonomous; a true agent decides the order and chains results itself.

I. Interview Questions

Q1. What is tool binding, and which LLM method is used to perform it?

Q2. Explain the difference between an `AIMessage`'s `content` and its `tool_calls` attribute.

Q3. Does the LLM ever actually execute a tool during tool calling? Justify your answer.

Q4. What four fields make up a single entry in `AIMessage.tool_calls`?

Q5. What is the difference between calling `tool.invoke(tool_call["args"])` and `tool.invoke(tool_call)`?

Q6. What is a `ToolMessage`, and why must it be appended back to the messages list?

Q7. Describe a scenario where a chained tool call would produce an incorrect result without `InjectedToolArg`.

Q8. How does `Annotated[float, InjectedToolArg]` change what the LLM includes in its tool call arguments?

Q9. Why is the real-time currency converter built in this chapter not considered an AI agent?

Q10. What single characteristic most distinguishes an autonomous AI agent from a manually orchestrated tool-using application?

F. Further Reading

- [LangChain official documentation](#) — Tool calling concepts and how-to guides
- [LangChain official documentation](#) — `InjectedToolArg` reference in `langchain_core.tools`
- [LangChain official documentation](#) — Message types (`HumanMessage`, `AIMessage`, `ToolMessage`, `SystemMessage`)
- [ExchangeRate-API documentation](#) — live currency conversion rate endpoints
- [LangChain official documentation](#) — AI Agents (preparation for the next chapter)